

Comparison of Agentic AI Evaluation Toolkits

We compare **DeepEval**, **RAGAs**, **Gorilla (BFCL)**, **Evidently**, and **OpenCompass** – five open-source Python toolkits for evaluating agentic LLM systems. Our focus is on agentic-AI metrics: task execution quality (success, accuracy of actions), operational efficiency (path length, cost, throughput), reasoning explainability, safety/adherence, and consistency/reliability. All tools support Python and can call LLMs (including via OpenAI’s API) and be deployed in Docker/Kubernetes environments. The following table summarizes how each toolkit addresses these criteria, with evidence from their documentation and source code.

Package	Task Execution Quality (outcome success, tool accuracy)	Operation Efficiency (path length, compute cost, scalability)	Reasoning & Explainability (trajectory accuracy, interpretability)	Safety/Adherence (guaranteed adherence, safety)
DeepEval (confident-ai/deepeval)	Provides explicit agentic metrics: Task Completion and Tool Correctness ¹ for measuring whether an agent achieved its goal and invoked tools correctly. It also offers numerous LLM-output metrics (RAG metrics like answer relevancy, faithfulness, contextual recall/precision, RAGAs, etc.) ² ¹.	Focus is on output quality rather than speed. It does not measure path length or compute throughput. (However, it integrates with any CI/CD pipeline, making it easy to include in production testing ³.)	Includes advanced metrics (e.g. Deep Acyclic Graph “DAG” analysis ⁴) but does not explicitly produce human-readable reasoning traces. It focuses on quantitative scores rather than explainable trajectories.	Includes metrics in “reasoning” team for safety checks, toxicity, SQL injection, etc. ⁵. It guarantees unsafe actions (guaranteed safety).

Package	Task Execution Quality (outcome success, tool accuracy)	Operation Efficiency (path length, compute cost, scalability)	Reasoning & Explainability (trajectory accuracy, interpretability)	Safety Adherence (guardrail adherence, safety)
RAGAs (explodinggradients/ragas)	– Specializes in RAG pipelines. Provides component-level metrics: context precision/recall (retriever quality) and faithfulness/answer relevancy (generator accuracy) ⁶. It also supports agentic metrics – e.g. ToolCallAccuracy and AgentGoalAccuracy for multi-turn agent workflows ⁷ ⁸.	– Does not natively measure resource use or path length. Emphasis is on output quality. (It can generate synthetic test data and scale evaluations, but per-call cost or latency is not tracked.)	– Uses LLMs as judges for many metrics (LLM-as-judge style), but does not produce explicit reasoning logs. Focus is on final answer quality (answer correctness/semantic similarity).	– No adversarial guardrails. Its score mostly reflects function correctness (Adversarial evaluation could be scripted to cover a corner case).
Gorilla (BFCL) (gorilla.cs.berkeley.edu, bfcl-eval)	– Focused on function-calling / tool-use evaluation. The BFCL benchmark measures an LLM’s accuracy in selecting and invoking functions (tools) correctly ⁹. It reports overall accuracy of function calls, reflecting end-to-end agent success in tool use. Multi-turn interactions are included (BFCL-v3/4) for holistic agentic tasks.	– Explicitly tracks compute cost and latency for model queries. The BFCL leaderboard shows estimated cost (USD) and latency (s) for entire benchmarks ¹⁰. This directly addresses operational efficiency of agents.	– Captures multi-step function-calling workflows, but does not provide human-readable reasoning traces. It scores whether the final sequence of calls was correct (strict order matching).	– No adversarial guardrails. Its current focus is on correctness of function calls, not on malicious function scores or incorrect outputs.

Package	Task Execution Quality (outcome success, tool accuracy)	Operation Efficiency (path length, compute cost, scalability)	Reasoning & Explainability (trajectory accuracy, interpretability)	Safety Adherence (guardrails, adversarial safety)
Evidently (evidentlyai/evidently)	– A general ML/LLM observability toolkit. Offers text evaluation metrics (sentiment, grammar, length, etc.) and can use LLM-based judges for answer correctness (via “presets” like TextEvals). However, it has no built-in notion of multi-step agent success; it treats LLM output as text to be analyzed ¹². ¹³ .	– Not designed for efficiency metrics. It does support running reports/presets continuously (monitoring dashboards) so it can scale to production data. But it does not measure model cost or path length.	– Provides analysis dashboards (reports) and can include LLM-as-judge metrics, but does not log the chain of reasoning. It can flag denials or off-topic content via descriptors, aiding interpretability.	– Includes qualitative content e.g. d for de denia (“so bias, can s simpl guard “den detec Howe speci adver testin in.

Package	Task Execution Quality (outcome success, tool accuracy)	Operation Efficiency (path length, compute cost, scalability)	Reasoning & Explainability (trajectory accuracy, interpretability)	Safety Adherence (guaranteed adversarial safety)
OpenCompass (open-compass/opencompass)	– A broad LLM evaluation platform (100+ benchmarks). It includes an “Agent” category to evaluate tool-use and agent frameworks ¹⁶ . While it mostly focuses on standard NLP tasks (Q&A, reasoning, coding), it does support evaluating agents and tool invocation. Benchmarks like “CompassBench” can include agent scenarios.	– No built-in metric for efficiency (cost or latency). OpenCompass can evaluate models using different backends (OpenAI API or local runtimes) ¹⁷ , but it does not itself record resource usage.	– Rich in reasoning evaluation: supports complex reasoning/ coding benchmarks (math verification, multi-hop QA, etc.). Introduced a “GenericLLMEvaluator” (LLM-as-judge) for more nuanced assessment ¹⁸ . Does not produce explainable paths, but covers many reasoning skills.	– Has robust evaluation (“Robustness” category attacks ¹⁶ . test metrics adhere to underlying perturbations. However, guardrails against toxic outputs are not a primary

Each toolkit supports Python and can be containerized (Docker/K8s). All support calling external LLMs (including via the OpenAI API) for evaluation. DeepEval, RAGAs, Evidently and OpenCompass can use any LLM through their APIs (e.g. via environment variables or LangChain wrappers) ¹⁹ ¹⁷ . Gorilla’ s BFCL is focused on function-calling and also offers a pip-installable CLI (`bfcl-eval`) ²⁰ that can use OpenAI or local LLM endpoints.

The table above summarizes strengths:

- **DeepEval** excels at agentic accuracy (task completion, tool correctness) and built-in safety/red-teaming checks ¹ ⁵ , making it suitable for verifying agent behavior and safety in CI/CD.
- **RAGAs** provides comprehensive RAG and agentic metrics (context/answer quality, tool usage accuracy) ⁶ ⁷ , along with automatic test data generation, but does not directly measure efficiency or adversarial robustness.
- **Gorilla (BFCL)** is specialized for tool-use accuracy and reports cost/latency, giving insight into efficiency of agentic workflows ¹⁰ . It is the go-to for function-calling benchmarks but has limited general LLM analysis beyond that.
- **Evidently** is a general-purpose evaluation/monitoring framework with 100+ metrics ¹⁵ . It is strong on production monitoring (dashboard, drift, metrics logging) and flexible descriptive tests, but not focused on multi-step agent tasks.

- **OpenCompass** covers a broad range of benchmarks (including agentic and reasoning tasks) ¹⁶ ¹⁸ . It allows large-scale comparison of models but does not itself manage model serving or efficiency measurements.

Each tool's documentation provides details on its capabilities (see sources above). The comparative strengths guide which toolkit best fits a given production use case. For example, for strict task success and safety testing of an agent, DeepEval and Gorilla/BFCL offer targeted support; for ongoing performance monitoring, Evidently is suitable; and for benchmarking across diverse reasoning tasks, OpenCompass provides extensive coverage ¹ ¹⁰ ¹⁵ ¹⁶ .

Sources: Tool documentation and README files as cited above (DeepEval ¹ ⁵ , RAGAs ⁶ ⁷ , Gorilla/BFCL ⁹ ¹⁰ , Evidently ¹⁵ ¹⁴ , OpenCompass ¹⁶ ¹⁸). All information is drawn from these open-source project references.

¹ ² ³ ⁴ ⁵ ¹⁹ GitHub - confident-ai/deepeval: The LLM Evaluation Framework

<https://github.com/confident-ai/deepeval>

⁶ Evaluating RAG Applications with RAGAs | by Leonie Monigatti | TDS Archive | Medium

<https://medium.com/data-science/evaluating-rag-applications-with-ragas-81d67b0ee31a>

⁷ ⁸ Agentic or Tool use - Ragas

https://docs.ragas.io/en/latest/concepts/metrics/available_metrics/agents/

⁹ ¹⁰ Berkeley Function Calling Leaderboard (BFCL) V4

<https://gorilla.cs.berkeley.edu/leaderboard.html>

¹¹ ²⁰ bfcl-eval • PyPI

<https://pypi.org/project/bfcl-eval/>

¹² ¹³ ¹⁴ ¹⁵ GitHub - evidentlyai/evidently: Evidently is an open-source ML and LLM observability framework. Evaluate, test, and monitor any AI-powered system or data pipeline. From tabular data to Gen AI. 100+ metrics.

<https://github.com/evidentlyai/evidently>

¹⁶ ¹⁷ ¹⁸ GitHub - open-compass/opencompass: OpenCompass is an LLM evaluation platform, supporting a wide range of models (Llama3, Mistral, InternLM2, GPT-4, LLaMa2, Qwen, GLM, Claude, etc) over 100+ datasets.

<https://github.com/open-compass/opencompass>